Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version XX Month, XXXX.

Digital Object Identifier 10.1109/XXXX.2022.1234567

Assessing Privacy Leakage in Android App Logs: An Empirical Study of Recorded and Real-World Cases

Zhiyuan Chen[1], Vanessa Nava-Camal[1], Tiash Roy[1], Yiming Tang[1], Xueling Zhang[1], Derui Zhu[2], Andy Meneely[1], and Weiyi Shang[3]

[1] Department of Software Engineering, Rochester Institute of Technology, Rochester, NY, USA
[2] TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany
[3] Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada

Corresponding author: Derui Zhu (email: derui.zhu@tum.de).

ABSTRACT

Android applications generate detailed logs for debugging and monitoring, but these logs may also contain sensitive user information, including Personally Identifiable Information (PII). To investigate privacy leakage in Android app logs, this study examines both publicly reported log-related vulnerabilities and privacy-relevant information observed in real-world Android app logs. We analyzed 68 vulnerabilities reported in the Common Vulnerabilities and Exposures (CVE) database, covering 99 privacy information cases. Our results show that 81.54% of vulnerabilities with available user-interaction metrics required no user interaction, 91.07% of those with attack-complexity metrics had low attack complexity, and 42.65% of the studied vulnerabilities had a network attack vector. To investigate privacy leakage in practice, we constructed an Android app log dataset covering 300 diverse and popular apps. We combined traditional keyword-based detection with LLM-based approaches to identify privacy-relevant information in the collected logs. We detected 8,455 occurrences of privacy-relevant information in logs collected during the execution of 299 apps (99.67%), with 19.66% of these occurrences associated with third-party sources. These findings highlight the widespread presence of privacy-relevant information in Android app logs and the need for developers to strengthen privacy protection in their logging practices.

INDEX TERMS Android, log, privacy leakage

I. Introduction

Mobile technology is intricately woven into humanity's daily activities, such as entertainment, business, and social interaction. Consequently, mobile apps often store vast amounts of users' private data, including Personally Identifiable Information (PII), which must be protected against unauthorized disclosure, as required by various privacy protection regulations such as the GDPR [1], CCPA [2] and PIPL [3]. The leakage of sensitive user data from mobile apps can lead to severe consequences, such as identity theft and other privacy harms [4], [5]. App developers have both a legal and ethical obligation to safeguard users' private information from unauthorized access and leakage [6], [7].

During runtime, mobile apps generate numerous logs that serve critical roles in software engineering tasks such as troubleshooting, performance monitoring, and system analysis. System logs are an invaluable resource for diagnosing problems, understanding workflow, and improving the reliability of complex software systems [8]–[10]. By analyzing logs, developers and researchers can understand the state of the system and detect system anomalies. Log analysis can help optimize or debug system performance [11], [12].

Although app logs are useful for development and operations, they may also inadvertently contain sensitive user data. If exposed, private information could be harvested and misused by malicious attackers. In most cases, an attacker needs root or elevated privileges to access system logs, as these are usually protected by Android's access control mechanisms. However, log data may also be collected or transmitted by apps or third-party SDKs, creating potential exposure beyond the device. Additionally, attackers who gain sufficient access through a separate device compromise may be able to read the logs. Therefore, the local nature of logs does not guarantee their confidentiality.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

Log-related privacy vulnerabilities have been reported in the past [13], [14], and the Android documentation highlights the importance of avoiding sensitive information in logs [15]. Therefore, an evidence-based approach to understanding and characterizing privacy risks in Android app logs is necessary.

There have been many studies [16]–[18] investigating privacy leakage in Android apps, and many of them have attempted to assess the severity of such leakage using various metrics [19], [20]. However, these studies primarily focus on general Android app behaviors rather than app logs. Further empirical investigation is needed to examine publicly reported log-related vulnerabilities alongside privacy-relevant information observed in Android app logs. Understanding these two sources of evidence can help developers better comprehend the privacy risks associated with logging practices.

In recent years, large language models (LLMs) have shown great potential in automating complex text classification tasks, including the detection of sensitive information. Unlike fixed keyword lists, LLMs may recognize diverse textual patterns without requiring every variation to be manually specified [21], [22]. Additionally, Android application logs combine structured metadata with free-form messages, which can make sensitive-information detection challenging for fixed keyword lists [23], [24]. Therefore, we investigate LLM-based detection as a complement to traditional keyword-based detection in Android application logs.

The goal of this work is to characterize privacy risks in Android app logs from two complementary perspectives: publicly reported vulnerabilities and privacy-relevant information observed in collected runtime logs. Specifically, we mined Android log-related vulnerability records from the CVE dataset[1] to analyze publicly reported vulnerabilities. We also analyzed privacy-relevant information in logs collected during Android app execution using a detection strategy that combines an LLM-based approach with a traditional keyword-based strategy.

We analyzed 68 CVE records, including their available CVSS severity scores and metrics, such as attack vector and authentication-related requirements. To examine privacy-related information in practice, we created a log dataset covering 300 Android apps and identified 8,455 occurrences of privacy-relevant information in logs collected during the execution of 299 apps (99.67%), with 19.66% of these occurrences associated with third-party sources, including apps and libraries.

The contributions of this paper can be summarized as follows:

- **Dataset.** We collected a large Android app log dataset covering 300 diverse and popular apps, which can facilitate future log analysis and security research.
- **Impact of Vulnerabilities.** We characterized publicly reported Android log-related vulnerabilities using CVSS metrics to examine their reported severity and exploitability characteristics.
- **Privacy Leakages in Practice.** We examined privacy-relevant information observed in Android app logs, which can help developers understand the types and frequency of information recorded during app execution.
- **LLM Methodology.** We examined the strengths and limitations of LLM-based privacy leakage detection strategies, showing that, in our experiments, they complement rather than replace traditional keyword-based detection.

a: Paper Organization.
Section II presents the background of this study. Section III presents our research questions. Section IV presents our experimental results, and Section V discusses our results. Section VI presents our related work, and Section VII presents our threats to validity. Finally, Section VIII concludes the paper.

II. Background

In this section, we define and discuss the significance of privacy leakage in Android app logs.

A. What is Privacy Leakage in Android App logs?

We use the official Android documentation as guidance for our definitions, as well as the Common Weakness Enumeration (CWE)[2]. The Android documentation describes a type of privacy leakage vulnerability in app logs, known as **Log Information Disclosure** [25], which refers to apps printing sensitive information into their logs. This is CWE-532: Insertion of Sensitive Information into Log File[3].

The documentation also emphasizes that PII is a type of sensitive information that is prohibited from being logged [25]. PII refers to information that can be used to distinguish or trace an individual's identity, either on its own or when combined with other information that is linked or linkable to a specific individual [26]. The Android documentation specifically emphasizes this concern to raise developers' awareness of privacy protection and help them prevent private information from being logged. Privacy leakage in logs is not merely a coding issue but a security and compliance concern. Sensitive information written into logs may be accessed by privileged applications, third-party SDKs, or attackers who exploit other vulnerabilities. If such information includes PII, its exposure may constitute unauthorized disclosure under data protection regulations such as GDPR, CCPA, or PIPL. These regulations require appropriate technical safeguards and data minimization. Therefore,

[1] https://github.com/CVEProject/cvelistV5

[2] cwe.mitre.org

[3] https://www.cvedetails.com/cwe-details/532/Insertion-of-Sensitive-Information-into-Log-File.html

logging sensitive user data can have both security and legal consequences.

B. An Example of Privacy Leakage in Android App Logs

The following text shows an example of privacy leakage in Android app logs that we obtained through our interaction with an Android app. The log includes a user's email, `ritresearch23@gmail.com`, which is considered PII and could allow malicious attackers to identify the user.

> 05-03 23:30:25.186 2150 2150 W JobScheduler: Job didn't exist in JobStore: d84a90 SyncManager #1000/686 @SyncManager@com.google.android.
> keep/com.google/ritresearch23@gmail.com:android

Android app logs tend to be comprised of two types of information. One is added by the logging framework, not by the developers when writing the logs. The other is customized by the developers, representing the information they expect to see during runtime. In this example, the colon immediately following "JobScheduler" separates two different types of content. The content before the colon is automatically added by the logging framework and does not appear in the logging code, while the content after the colon is written by the developers in the logging code. The content automatically added by the logging framework typically does not contain privacy leakage, while privacy leakage often occurs in the content written by the developers. However, since there is no unified standard for what developers want to log, this leads to logs containing a wide variety of forms of private information and makes privacy leakage detection more challenging.

C. The Availability of Android App Logs.

One may assume that logs are only available locally, and therefore not easily accessible to malicious users. We do explore this in our analysis in Section IV through CVSS scores, but there are good reasons to not consider logs inherently protected.

Since Android 4.1, Google has restricted log reading permissions, allowing only privileged system apps to access system logs [15], which further enhances privacy protection. As a result, even if they are aware that logs may contain privacy leakage, they often lack a real sense of risk. However, according to the official documentation [15], Android supports a vast number of devices that may have **pre-loaded apps** with permissions to read system logs, and private information in logs could be collected by these apps. Therefore, the documentation does not recommend logging any private information in system logs. Additionally, many apps use **third-party SDKs** that automatically collect and send logs to remote servers for analysis, and such data transmission can further amplify the risk of user privacy leakage. Lastly, the protection of logs through permissions can be bypassed in the presence of a **separate vulnerability**, for example, malware with elevated privileges may extract sensitive data from logs. Logs are frequently backed up, synchronized, or stored in shared environments during development and testing. Once exported or aggregated outside the device, log contents may be exposed through misconfiguration or insufficient access control.

III. Research Questions

We aim to explore privacy leakage in Android app logs from two perspectives: (1) publicly disclosed log-related vulnerabilities recorded in CVEs, and (2) privacy leakage disclosed in real-world Android app logs. Our research questions are:

RQ1: What do publicly reported CVEs reveal about privacy leakage in Android app logs?

The Common Vulnerabilities and Exposures (CVE) project is a database that tracks publicly-known vulnerabilities and exposures across various software and hardware systems [27]. The CVE has over 270,000 entries. By mining CVE records related to Android app logs, we analyze how log-based privacy leakage has been documented and characterized within the vulnerability reporting ecosystem. As part of the CVE reporting process, vulnerabilities are assessed using the Common Vulnerability Scoring System (CVSS), which provides standardized severity metrics. We examine these records and their associated CVSS scores to understand how disclosed log-related privacy issues are evaluated in practice. The data for RQ1 consist of publicly disclosed vulnerabilities. However, CVEs only capture issues that have been reported and formally documented. To examine privacy leakage in practice , we further examine privacy leakage from real-world Android app logs:

RQ2: What privacy-relevant information appears in real-world Android app logs?

We study the actual log data in practice. Unlike CVE records, these logs reflect runtime behavior in practice and may contain privacy-relevant information that has not been formally reported as a vulnerability. Our goal is to systematically identify and analyze such cases to provide an empirical view of privacy leakage in Android app logs.

IV. Experiments

In this section, we present the empirical experiments conducted to address the research questions. For Research Question 1, we analyze Android log-related CVE records collected from the official CVE database, examining the types of privacy information involved and their corresponding CVSS scoring metrics. For Research Question 2, we constructed a real-world Android application log dataset from 300 popular apps and employed a multi-stage detection pipeline to identify privacy leaks in raw logs.

A. RQ1: CVE Records and Privacy Leakage Severity in Android App Logs

1) Data Collection and Analysis Method

We cloned the official CVE repository[4] and queried the CVE records using the keywords "android" and "log". We did not include additional privacy-related keywords (e.g., "PII" or "leakage"), which could bias the search toward explicitly labeled vulnerabilities. Due to the potential for false positives in keyword-based query results (e.g., records may contain "log" as part of "log in" rather than referring to logging), each candidate record was independently reviewed by two reviewers. In cases of disagreement, a third reviewer served as a tiebreaker.

With these data, we did the following:

- Identified the privacy information recorded in CVE records to determine whether they involve PII.
- Analyzed the CVSS scores of the CVE records. CVSS provides a numerical measure of vulnerability severity [28], based on metrics such as Access/Attack Vector, Attack Complexity, and, depending on the CVSS version, Authentication, Privileges Required, or User Interaction.

2) Results

We initially identified 78 CVE records for this study through keyword matching. After manual analysis, we filtered out 10 false positives due to semantic mismatches, resulting in a final dataset of 68 records. These 68 records constitute the set of CVEs identified by our keyword search and manual screening at the time of data collection. They are not a random sample, but the search may not capture every relevant Android log-related vulnerability. As some CVE records contain multiple cases of privacy information, we identify separate **cases** individually rather than at the CVE record level. In the end, we analyzed 99 privacy information cases across the 68 CVEs. Each case of privacy information was labeled with one of the following four labels: PII, non-PII, possible, or N/A. As shown in Table 1, we provide representative examples for each classification category. The "possible" label indicates cases where:

- the privacy information could be considered PII under certain conditions. For example, whether text message content qualifies as PII depends on whether it contains information that can identify a user; or
- it is generally not considered PII but could lead to the leakage of PII. For example, Attention (AT) commands and parameters are not considered PII, but they could allow a malicious attacker to obtain the user's outgoing and incoming calls and text messages [29].

[4]https://github.com/CVEProject/cvelistV5

The "N/A" label indicates that a CVE record describes a log-related issue without specifying the type of information potentially exposed.

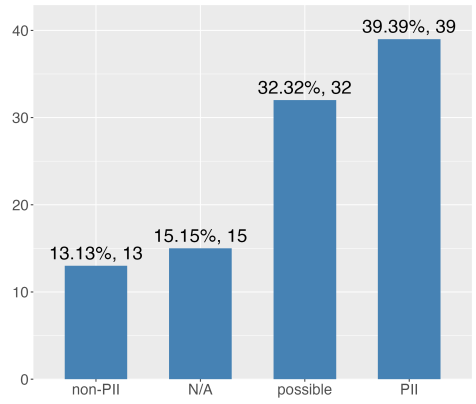


FIGURE 1. PII classification of privacy-information cases extracted from CVE records related to Android app logs.

According to Figure 1, PII is the largest single category of privacy information described in the CVE records (39.39% of the 99 cases). The potential impact of such information depends on its sensitivity and the circumstances of exposure. Additionally, non-PII information accounts for 13.13% of cases. For example, CVE-2017-18140 [30] (see below) describes an attempt to print an "RIL token ID" to a debug log while processing a call disconnection. However, the reported vulnerability concerns a potential use-after-free condition; the CVE description does not establish that logging the identifier itself enables unauthorized disclosure or a man-in-the-middle (MITM) attack.

> *...when processing a call disconnection, there is an attempt to print the RIL token-id to the debug log. If eMBMS service is enabled while processing the call disconnect, a Use After Free condition may potentially occur.*

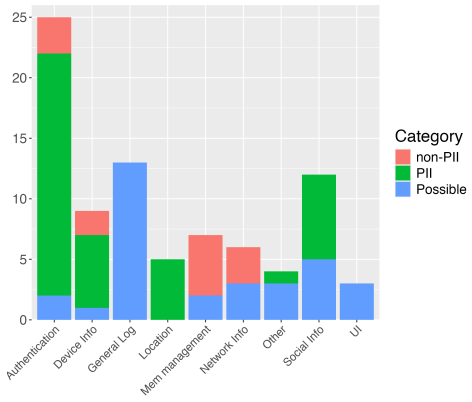


FIGURE 2. Categories of privacy information and PII classifications across cases extracted from CVE records related to Android app logs.

Figure 2 illustrates the categorization of privacy information in Android app logs recorded in the CVE dataset. For

TABLE 1. Representative examples of privacy information classification

Category	Example	Rationale
PII	IMEI / MAC address	Unique device identifiers that can be linked to a specific user or device.
PII	Authentication credentials (e.g., password, login credentials)	Enables direct account compromise and identity misuse.
Non-PII	RIL token-id	Internal system identifier not directly linked to user identity.
Non-PII	Kernel pointer / memory address	Technical system-level information without direct user identifiability.
Possible	Session ID	May enable account takeover or indirect identification depending on context.
Possible	Text message content	May contain sensitive user-identifiable information depending on message content.
Possible	Log file metadata	Not inherently identifying, but may facilitate correlation with other sensitive data.
N/A	Not specified in CVE record	CVE mentions log-related leakage but does not describe the exact type of leaked information.

example, the "authentication" category includes information related to authentication, such as passwords and usernames. As shown in the figure, PII is mainly associated with specific categories, suggesting that related logging code may warrant closer review. On the other hand, "possible" cases are more widely distributed and may require additional context to determine whether the information is personally identifying. Non-PII cases are concentrated in a few categories. These distributions do not, by themselves, establish the severity of individual vulnerabilities.

We then studied the CVSS scores of CVE records. Some CVE records include scores from multiple CVSS versions. For each CVE record, we collected the CVSS Base Score and its associated metrics from the NVD database. CVSS v2.0 scores were used as the primary reference; where CVSS v2.0 data were unavailable, CVSS v3.0 scores were used instead. The definitions and scales of some metrics differ across versions, so cross-version summary statistics should be interpreted with caution. The average CVSS Base Score is 4.27, the impact score is 4.04, and the exploitability score is 5.45. CVSS Base Scores range from 0 to 10, with higher scores indicating more severe vulnerabilities under the corresponding version. A Base Score between 4.0 and 6.9 is typically classified as medium severity [28]; however, the pooled averages do not represent fully comparable measurements across versions. In particular, the impact and exploitability subscores should not be interpreted as directly comparable across CVSS versions. Table 2 presents the severity distribution of the studied vulnerabilities. Medium is the most common severity category (29 of 68 records), while 12 records are classified as high.

CVSS scores are calculated from metrics describing vulnerability impact and exploitability. These components provide additional context beyond the Base Score. Below, we summarize the available CVSS metrics for the 68 CVE records. For some metrics, the denominator is less than 68 because the metrics are version-specific or unavailable for certain records:

TABLE 2. A summary of the severity of CVE records.

Severity	Records
HIGH	12
MEDIUM	29
LOW	27

- For 29 of 68 records, the **Attack/Access Vector** is "Network". This indicates that the reported vulnerability can be exploited over a network, but it does not necessarily mean that an attacker can directly retrieve local logs.
- For 29 of 68 records, the available CVSS metrics indicated **Authentication: None**. This characterizes an exploitation prerequisite; it does not indicate the difficulty of forensic investigation. CVSS v3 uses a separate Privileges Required metric.
- 64 of 68 records indicated an impact on **Confidentiality**.
- 12 of 58 records with available metrics indicated an impact on **Integrity**, suggesting potential effects on data or system operations.
- 51 of 56 records with available **Attack Complexity** metrics were marked as "LOW", suggesting that exploitation generally does not depend on specialized conditions. Low attack complexity contributes to exploitability but does not alone determine overall severity.
- 53 of 65 records with available **User Interaction** metrics were marked as "NONE", indicating that exploitation does not require action by a separate user.

From the above results, we observe that although medium is the most common severity category among these CVE records, the individual metrics reveal exploitability characteristics not fully captured by the overall severity distribution. Some records have network-based attack vectors, low attack complexity, or no user interaction requirement, and most indicate a confidentiality impact. These metrics characterize the reported vulnerabilities, rather than the sensitivity or accessibility of logged information alone.

The goal of RQ1 is not to redefine or recompute vulnerability severity, but to examine how privacy leakage in Android app logs is characterized within the existing vulnerability reporting ecosystem. CVSS scores represent standardized assessments of reported vulnerabilities, not direct measures of privacy harm from logged data. While CVSS scores may be updated over time, our analysis focuses not only on the numeric Base Score but also on the characteristics of the vulnerabilities.

Summary of RQ1

PII was the **largest single category** among the privacy-information cases described in the CVE records, while some cases involved non-PII information or information whose identifiability depends on context. Several CVE records had **network-based attack vectors**, while many showed low **attack complexity** or required no **user interaction**. These metrics describe the reported vulnerabilities rather than direct exposure of the logged information.

B. RQ2: Analyzing Privacy Leakage in Real-World Android App Logs

In this RQ, a "privacy leakage case" denotes an occurrence of privacy-related information detected in collected logs, not a confirmed unauthorized disclosure or a distinct vulnerability. We distinguish directly identifying and location-related information from device characteristics, such as manufacturer and model.

1) Log Collection and Detection Procedure

Phase 1. Log collection. We selected 300 popular Android apps from AndroZoo[5] based on download counts and category diversity, including social media, communication, finance, health, productivity, entertainment, and utilities. We used AndroZoo's category labels; app selection and log collection took place from May 3 to May 26, 2025. The collected logs reflect the app versions available at that time. We installed the apps using Android Debug Bridge (adb) [31] and collected Logcat records while one tester interacted with each app for approximately 30 minutes on a Google Pixel 5a. In total, we collected 8,012,451 log records (26,708 per app-session file on average). Because Logcat records device-level activity, entries observed during a test session cannot necessarily be attributed to the app under test; app counts below refer to sessions with matching entries.

Phase 2. Privacy-related information detection. As illustrated in Figure 3, we first searched for values from an experimental user profile using manually selected keywords informed by prior work [23], [32]. We then used ChatGPT (GPT-4o mini) in two complementary steps: direct detection, which identifies candidate privacy-related entries without supplied examples, and prompt-enhanced detection, which provides known examples and their expected outcomes using few-shot prompting. The logs were submitted to ChatGPT in consolidated batches containing 30 log files. We subsequently used the expanded keyword list in scripts to search the collected logs and obtain the final results. Combining the strategies helps address both the limited coverage of initial keywords and the omissions and false positives of LLM detection. The profile was constructed for the experiment, rather than from recruited participants; device-level logs may nevertheless contain unrelated identifiers.

Phase 3. Third-party-source analysis. Since the implementation of some third-party libraries is not fully visible to app developers, how these libraries handle user data may be difficult to inspect. As a result, privacy leakage issues may remain unnoticed by developers. Developers may have limited control over third-party library behavior. Once user information is collected, its subsequent handling may depend on the third-party provider. Android's data-use guidance addresses applicable collection or sharing of user data by third-party SDKs and libraries [33]; a local log entry alone does not establish such collection or sharing. We examined Logcat metadata, particularly recognizable tags such as `CleverTap`, to identify entries associated with third-party components. For example, the tag in `D CleverTap:...` is suggestive of the CleverTap library, but it does not independently establish which component emitted each entry or whether any data were transmitted. We therefore distinguish third-party libraries from independently identified third-party apps in interpreting these results.

2) Results

a: Basic keyword detection.

The initial strategy searched for known values in an experimental user profile. Table 3 reports 514 privacy-related matches in 81 of the 300 app sessions (27%). Email (194 matches) was the largest category, followed by manufacturer/model (190). Coarse-grained location represents state/province-level strings and fine-grained location represents city-level strings; neither necessarily establishes the user's actual location. The original keyword list contained only one exact GPS coordinate and could miss other location values or previously used identifiers. GPS coordinates can reveal a more precise position if they correspond to the user

[5] https://androzoo.uni.lu/

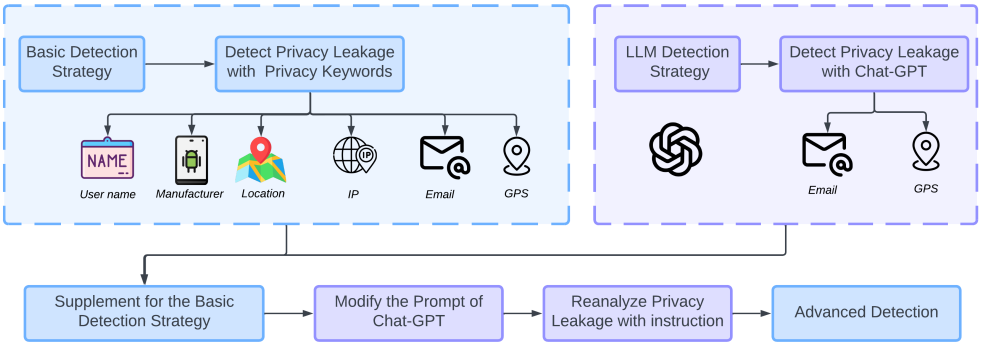


FIGURE 3. **Workflow for privacy leakage detection.**

TABLE 3. **Privacy-related matches under basic keyword detection.**

Info. Type	Profile value	Sess.	Matches
Email	ritresearch23@gmail.com	45	194
Manufacturer/model	Google Pixel 5a	19	190
User name	rit research	24	82
IP address	68.180.86.24	6	18
Coarse location	New York	10	13
Fine location	Rochester	9	11
GPS	43.0838693, -77.6799839	3	6
Total		81	514

TABLE 4. **Privacy-related matches retained under direct LLM detection.**

Information type	Sessions	Matches
Email (initial address)	7	68
Email (previous address)	2	32
GPS-coordinate patterns	14	37
Total matches	–	**137**

or device. Exposure of such information to an unauthorized party could create a risk of tracking.

b: Direct LLM detection.

ChatGPT flagged 32,863 candidates, of which 137 were retained after manual examination and 32,726 were rejected. The manual examination involved grouping the 32,863 entries flagged by ChatGPT into 4,295 categories according to the identified information, with a representative log entry for each category. This grouping organized the logs before validation and reduced the review effort. Two independent reviewers then examined the representative entries. First, they assessed whether ChatGPT had identified the information correctly. Next, they determined whether the information constituted a privacy-related match under our study definition. In cases of disagreement, a third reviewer acted as a tiebreaker. To facilitate examination, logs sharing the same templates were reviewed using representative entries. Cohen's kappa was 0.95 for correctness assessment and 0.81 for privacy-related classification. Table 4 details the retained matches, which comprise two email addresses and GPS-coordinate patterns.

The second email address had been associated with the device but was absent from the initial keyword list, demonstrating how direct LLM detection can recover overlooked values. Conversely, for the first email address, the LLM retained only 68 matches across 7 sessions, compared with 194 matches across 45 sessions under basic keyword detection. The strategies thus yielded different coverage even for the same known value. Session counts in Table 4 are per category and should not be summed to infer a distinct-session total.

The high number of false positives suggests that ChatGPT may rely on superficial patterns of sensitive information and misclassify unrelated content. For example, in the following case, ChatGPT incorrectly identifies a numeric token as a phone number because its length matches that of a typical phone number.

```
05-17 06:36:45.618 1176 1277 I
Delight5Decoder: Delight5DecoderWrapper.
loadLanguageModel():729
loadLanguageModel() : 1, version
[2023101000]
```

According to the discussion above, it can be concluded that ChatGPT should not replace the basic detection strategy, but should rather serve as a supplement.

c: Prompt-enhanced detection.

Prompt enhancement flagged 6,004 candidates, of which 5,133 were retained after manual verification and 871 were rejected. Since our log generation process did not require keeping the device in one location throughout the collection period, using just one GPS coordinate was insufficient, and ChatGPT helped us find more GPS coordinate patterns. As Table 5 shows, the initial search found 6 GPS matches in 3 sessions, direct LLM detection found 37 in 14 sessions, and prompt-enhanced detection found 732 in 217 sessions. Only one match overlapped between basic and direct detection, whereas prompt-enhanced detection covered all six basic GPS matches. These results illustrate both the additional pat-

TABLE 5. GPS-coordinate matches across detection strategies.

Detection strategy	Sessions	Matches
Basic keyword detection	3	6
Direct LLM detection	14	37
Basic–direct overlap	1	1
Prompt-enhanced detection	217	732
Basic–enhanced overlap	3	6

TABLE 6. Prompt-enhanced detection results, excluding GPS matches.

Info. Type	Sessions	Matches
Manufacturer/model	239	3,889
Email (first address)	45	181
Email (second address)	96	218
User name	23	75
IP address	5	17
Coarse-grained location	9	12
Fine-grained location	7	9

TABLE 7. Privacy-related matches from the final detection strategy. Full matched values are provided in the replication package.

Info. Type	Sessions	Matches
Email (first address)	45	194
Email (second address)	96	218
Manufacturer/model	297	7,187
User name	24	82
IP address	6	18
Coarse-grained location	10	13
Fine-grained location	9	11
GPS	217	732
Total	299	8,455

terns found with LLM assistance and the potential omissions of direct LLM detection.

The category-level results of prompt-enhanced detection (excluding GPS) are summarized in Table 6. Device manufacturer/model information accounts for most of these matches (3,889), while the expanded strategy also identified both email addresses. The seven non-GPS categories contain 4,401 matches; together with 732 GPS matches, they yield 5,133 manually retained results. The stage-by-stage comparison also shows that more matches do not necessarily imply uniform gains across information types. For example, the first email decreased from 194 basic matches to 181 prompt-enhanced matches, and user-name matches decreased from 82 to 75. These omissions help explain why we subsequently searched the full log dataset with the expanded keyword list rather than treating the prompt-enhanced output alone as the final result.

Direct detection rejected 32,726 of 32,863 flagged candidates (99.58%), whereas prompt-enhanced detection rejected 871 of 6,004. This is a 97.34% reduction in the number of rejected candidates between the two candidate sets, not an independently established false-positive rate over all log records. The replication package retains the original matched values and complete representative log examples.

d: Final detection results.

Using the expanded keyword list to search the log dataset, we identified 8,455 privacy-related matches during 299 of 300 app sessions (99.67%), or 28.18 matches per studied session (Table 7). This final count is distinct from the 5,133 matches retained during prompt-enhanced detection.

Manufacturer/model information accounted for 7,187 matches (85.00%) across 297 sessions. As all apps were tested on the same Google Pixel 5a, these entries do not establish that 297 apps disclosed users' identities. Excluding this category leaves 1,268 matches; Table 7 does not determine how many distinct sessions contain those matches. The remaining categories include 412 email-address matches and 732 GPS-coordinate matches across 217 sessions. Email addresses are generally PII, while GPS patterns warrant closer examination to determine whether they refer to the user or device and which component emitted them.

The model name appeared in several forms (e.g., `Google Pixel 5a`, `Google Pixel`, `Pixel 5a`, and `Pixel`). A general keyword such as `Pixel` is ambiguous: it also appears in unrelated strings including `BLUE PIXEL` and `physicalPixelDisplaySizeRatio`. In contrast, a field such as `DeviceName=Pixel%205a` supplies device-model context. These examples motivate checking the context of manufacturer/model matches rather than treating every appearance as a privacy disclosure; manufacturer/model information is not inherently PII, although it may contribute to profiling when combined with other data.

e: Third-party-source results.

Table 8 lists 1,662 privacy-related matches (19.66% of the total) associated with recognizable third-party sources. This category includes third-party apps such as MileIQ, not only libraries. The top four listed sources account for 76.29% of these matches, including 690 associated with MileIQ. These counts indicate associations in the collected logs, not confirmed transmission or exclusive responsibility of third-party libraries.

TABLE 8. A summary of privacy-related matches associated with third-party libraries and apps.

Third-party source	Matches
MileIQ	690
AdobeExperienceSDK	331
AppsFlyer	128
Mixpanel	119
Apptimize	80
CleverTap	67
LaunchDarklySdk	50
OneSignal	42
HealthLife	28
Braze	28
Others	99
Total	1,662

Summary of RQ2

Our combined detection strategy identified 8,455 privacy-related matches in logs collected during 299 app sessions. Device manufacturer/model information accounted for 85.00% of the matches, while other categories included email addresses and GPS-coordinate patterns. LLM-assisted detection supplemented keyword search but produced false-positive candidates requiring review. We also identified 1,662 matches associated with third-party sources, including both apps and libraries; neither these associations nor the app-session counts establish unauthorized disclosure.

V. Discussion

In this section, we discuss the results of our study and present several conclusions drawn from the discussion.

Conclusion 1: Android logs may contain PII despite Android's privacy guidance. The official Android guidelines provide guidance regarding privacy protection in Android logs [15], [25]. However, in practice, the content recorded in Android logs may include privacy-related information. For example, the Android documentation [15], [25] advises against logging PII, yet we identified log entries containing email addresses and GPS-coordinate patterns. These findings warrant developers' attention and reviews of logging practices, although the presence of such information alone does not establish unauthorized disclosure or attribute every entry to the app under test.

Conclusion 2: CVE records and runtime logs provide complementary perspectives on privacy-related logging. In our previous RQs, we studied two sources of evidence about privacy-related logging: publicly reported vulnerabilities recorded in CVEs and privacy-related information observed in collected Android logs. These sources describe different phenomena and use different units of analysis (CVE records versus log occurrences), so their case counts cannot be directly compared. First, CVEs document formally reported vulnerabilities, whereas runtime logs reveal privacy-related values observed in the tested environment, which do not necessarily correspond to known vulnerabilities. Second, the CVSS scores characterize the severity and exploitability of the reported vulnerabilities, rather than the sensitivity of individual log entries. Although medium is the most common CVSS severity category in our CVE dataset, this does not determine the privacy risk of information observed in runtime logs. In our study, privacy-related matches were observed during 299 out of 300 app execution sessions, but 7,187 of the 8,455 matches (85.00%) involved the manufacturer/-model of our common test device. These matches should not be interpreted as confirmed unauthorized disclosures by 299 apps. Third, CVE disclosures concern specific documented vulnerabilities and cannot fully characterize other privacy-related information appearing in logs during app execution. These differences motivate examining both publicly reported vulnerabilities and collected runtime logs to obtain complementary evidence about privacy-related logging.

Conclusion 3: Logging levels do not consistently reflect the sensitivity of information recorded in Android logs. We examine logging levels to understand how log messages are categorized for diagnostic purposes, rather than to directly measure developers' attention to privacy risks. This information can be extracted from the metadata in the logs. For example, the "D" in the Android log below indicates that this log is at the DEBUG level [25]. Logging levels help developers filter log information during runtime, but whether lower-level messages are displayed depends on the logging configuration; they are not necessarily stripped at runtime. Android uses different logging levels to categorize the detail and severity of log entries, helping developers filter and prioritize what information to view during runtime. Higher logging levels like ERROR and WARN generally indicate more critical diagnostic events, while lower levels such as DEBUG, INFO, and VERBOSE are typically used for less severe information but may still contain sensitive data. These levels assist in managing log output, but do not directly indicate the sensitivity of the information recorded.

```
05-18 23:44:05.736 31269 31339 D CleverTap:...
```

According to Table 9, the total number of ERROR and WARN cases is 5,667, which accounts for 67.03% of the total cases. However, 5,334 of these matches contain manufacturer/model information, which accounts for 7,187 of the 8,455 matches overall. Excluding manufacturer/model, only 333 of the remaining 1,268 matches (26.26%) occur at ERROR or WARN levels. Therefore, the overall percentage is largely driven by device-model entries and does not establish that developers have given privacy-related information greater attention.

The table also suggests that logging levels do not consistently reflect the sensitivity of recorded information. For example, GPS coordinates can reveal more sensitive location information than manufacturer/model details, yet GPS-related entries mostly occur at lower logging levels (INFO and DEBUG), while most manufacturer/model entries occur at WARN. Even among different types of PII, there is no consistent pattern: for example, entries containing email addresses tend to appear at higher logging levels (ERROR and WARN), whereas GPS-related entries behave differently. Therefore, the sensitivity of privacy-related information in our dataset does not consistently align with logging levels; these observations do not establish how developers prioritize privacy protection.

TABLE 9. A summary of logging levels for Android log entries containing privacy-related matches.

Info. Type	E	W	I	D	V
Manufacturer	427	4,907	800	614	439
Email	179	137	14	49	33
User name	6	0	30	32	14
IP address	3	0	13	2	0
Coarse-grained location	5	0	2	0	6
Fine-grained location	3	0	2	6	0
GPS	0	0	371	355	6
Total	623	5,044	1,232	1,058	498

VI. Related Work

Privacy leakage in Android devices is a critical problem that has been acknowledged by many studies. Recent research indicates that mobile devices, especially Android devices, may leak large amounts of sensitive user information [16], [17]. As a result, this problem has attracted considerable attention from developers and researchers. For example, studies conducted on datasets such as Genome [18] and Mobile-Sandbox [34] have revealed that almost half of all malicious apps exhibit privacy leakage issues. In addition to malicious apps, even seemingly legitimate apps may violate users' privacy expectations. Slavin et al. [35] found 341 policy code inconsistencies in 477 apps, which indicates the significant gap between declared privacy policies and the actual behavior of the app. Yang et al. [36] note that data transmission represents another critical channel through which user privacy leakage may occur, highlighting the need to protect user privacy information during storage and transmission. Furthermore, a growing body of research has leveraged code analysis techniques, such as static and dynamic analysis, to systematically detect privacy leakage issues in Android apps. The findings of these studies highlight the importance of addressing privacy leakage in the Android apps. For example, Yang and Yang [37] utilize static taint analysis to examine the leakage of sensitive information in Android apps and successfully identified 145 cases of privacy leakage out of 1,750 analyzed apps. He et al. [38] present the critical role of the privacy leakage caused by third-party libraries in Android apps and used dynamic analysis techniques to investigate this issue, revealing that many third-party libraries access users' private information. In summary, the aforementioned studies confirm that privacy leakage in Android apps is not only a significant concern but also a severe problem. This alarming reality further inspires us to explore privacy leakage in Android app logs, an area that has received relatively little attention.

Although log analysis has been widely studied, fewer studies combine publicly reported log-related vulnerabilities with empirical observations of privacy-relevant information in Android app logs. Log study is a rapidly growing research area, with numerous papers being published each year. The growth of this field has been greatly facilitated by the release of a well-known log dataset named LogHub [39], which has been widely used in many log studies. For example, Huo et al. [40] propose SemParser, a semantic-based log parser, and conduct a comparative analysis on the LogHub dataset. Li et al. [41] propose a variable-aware log abstraction method and evaluate it on the LogHub dataset. High-quality log datasets are crucial for log research, as they ensure that studies are comprehensive and reliable. However, existing datasets do not necessarily provide the app coverage and runtime information needed for our analysis. Therefore, we constructed our own Android app log dataset. The work by Lyons et al. [42] is closely related to this study and investigates sensitive information recorded in the Android logging ecosystem. Their field study focuses on device-level logging behavior, access to system logs by preinstalled apps, and manufacturers' privacy practices. In contrast, our study examines logs collected during the execution of 300 Android apps and complements these observations with an analysis of publicly reported log-related vulnerabilities in CVE records.

Our earlier ASE 2024 poster [23] presented a preliminary empirical analysis of Android app logs, covering 83 apps and identifying 610 potential privacy leakage instances. The present study extends this line of research by examining a broader set of 300 apps, analyzing 68 CVE records and their severity metrics, and investigating privacy-relevant information associated with third-party sources. In addition, our previous IEEE Software article [43] explored the potential and limitations of LLM-based detection and prompt enhancement for identifying privacy leakage in Android app logs. Rather than treating LLM-based detection as an entirely new approach, this study incorporates it into a broader analysis of disclosed vulnerabilities and observed runtime logs. Relatedly, Zhao et al. [44] studied third-party Android logging frameworks using static taint analysis of 48,702 apps. While their work examines potential sensitive-data flows to logging APIs, our analysis examines privacy-relevant content present in collected runtime logs.

VII. Threats to Validity

In this section, we present the threats to validity of our study.

a: Internal Validity

In this work, we used only a single Android device to generate logs, rather than multiple devices. However, we collected all generated logs using the `Logcat` command line without any filtering or reduction, which ensures that our log dataset is comprehensive. Although using a single device means that the logs are associated with one specific Android system version, we used the latest versions of the apps when conducting our study. Therefore, the generated logs are up-to-date and representative of real-world usage.

b: External Validity

The apps we selected may not comprehensively represent the entire Google Play app ecosystem. However, to mitigate this limitation, we deliberately selected as many apps as possible, which ultimately resulted in 300 apps in our dataset. Our selection was based on app popularity (measured by the number of downloads), ensuring that the chosen apps cover the vast majority of those commonly used by real users, making them highly representative. It should be noted that our analysis of privacy leakage reflects the current status of these apps, and as apps continue to evolve, these data leakage status may change. However, since we downloaded and analyzed the latest versions of the apps at the time of our study, our results accurately represent the most up-to-date privacy leakage status in the Android app logs.

c: Construct Validity

It is challenging to create a fully comprehensive keyword list for detecting privacy leakage. Initially, we referred to existing studies [23], [32] and leveraged our domain knowledge to construct this list. However, due to the diverse nature of privacy information contained in Android app logs, it is nearly impossible to exhaustively cover all potential privacy leakage cases. To mitigate this issue, we employed LLM-based privacy leakage detection strategies, using two different LLM-driven strategies to expand and refine our initial keyword list. This process helped us build a more robust and comprehensive keyword list, enabling us to identify as many privacy leakage cases in Android app logs as possible.

VIII. Conclusion & Future Work

Mobile apps store large amounts of privacy-related information, and their runtime logs may contain sensitive data, including PII, raising privacy and security concerns. Although many studies have examined privacy leakage and assessed its severity, fewer have jointly investigated publicly reported log-related vulnerabilities and privacy-relevant information observed in Android app logs. This study examines these two complementary perspectives to better characterize privacy risks associated with logging practices. Medium severity is the most common CVSS category among the studied vulnerabilities. Specifically, 81.54% of vulnerabilities with available user-interaction metrics require no user interaction, 91.07% of those with attack-complexity metrics have low attack complexity, and 42.65% of the studied vulnerabilities have a network attack vector. We also constructed an Android app log dataset covering 300 apps and combined traditional keyword-based detection with LLM-based approaches to identify privacy-relevant information. We identified 8,455 privacy-related matches in logs collected during 299 app execution sessions (99.67%), with 19.66% of these matches associated with third-party sources, including apps and libraries. These matches include email addresses, GPS-coordinate patterns, and device manufacturer/model information. Although these observations do not necessarily establish unauthorized disclosure, they highlight the widespread presence of privacy-relevant information in Android logs and the importance of strengthening privacy protection in logging practices.

In future work, we plan to expand our Android app log dataset by incorporating automated UI testing to simulate more user interaction events, identify additional privacy-related information, and further investigate privacy leakage patterns in Android app logs.

IX. Declarations

a: Funding:
not applicable.

b: Ethical approval:
not applicable.

c: Informed consent:
not applicable.

d: Author Contributions:
The first author contributed to data analysis and writing. The second and third authors collected the data. The fourth author contributed to data analysis, writing, and supervision. The fifth, sixth, seventh, and eighth authors provided supervision.

e: Data Availability Statement:
To foster the open-source community, we have made the code and data public: https://github.com/AndroidAppLogsPrivacyLeakage/Privacy-LLM

f: Conflict of Interest:
The authors declare no conflict of interest.

g: Clinical trial number:
not applicable.

REFERENCES

[1] GDPR. General data protection regulation. [Online]. Available: https://gdpr-info.eu/
[2] S. of California Department of Justice. California consumer privacy act (ccpa). [Online]. Available: https://oag.ca.gov/privacy/ccpa
[3] NPC. Personal information protection law of the people's republic of china. [Online]. Available: https://personalinformationprotectionlaw.com/

[4] E. Bertino, "Data security and privacy: Concepts, approaches, and research directions," in 2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC), vol. 1, 2016, pp. 400–407.

[5] L. Rosenfeld, J. Torous, and I. V. Vahia, "Data security and privacy in apps for dementia: an analysis of existing privacy policies," The American Journal of Geriatric Psychiatry, vol. 25, no. 8, pp. 873–877, 2017.

[6] K. Shilton and D. Greene, "Linking platforms, practices, and developer ethics: Levers for privacy discourse in mobile application development," Journal of Business Ethics, vol. 155, no. 1, pp. 131–146, 2019.

[7] M. Hatamian, "Engineering privacy in smartphone apps: A technical guideline catalog for app developers," Ieee Access, vol. 8, pp. 35 429–35 445, 2020.

[8] Y. Zeng, J. Chen, W. Shang, and T.-H. Chen, "Studying the characteristics of logging practices in mobile apps: a case study on f-droid," Empirical Software Engineering, vol. 24, pp. 3394–3434, 2019.

[9] R. Spolaor, E. D. Santo, and M. Conti, "Delta: Data extraction and logging tool for android," IEEE Transactions on Mobile Computing, vol. 17, no. 6, pp. 1289–1302, 2018.

[10] R. Zhou, M. Hamdaqa, H. Cai, and A. Hamou-Lhadj, "Mobilogleak: A preliminary study on data leakage caused by poor logging practices," in 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2020, pp. 577–581.

[11] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," Commun. ACM, vol. 55, no. 2, p. 55–61, Feb. 2012. [Online]. Available: https://doi.org/10.1145/2076450.2076466

[12] J. Harty, H. Zhang, L. Wei, L. Pascarella, M. Aniche, and W. Shang, "Logging practices with mobile analytics: An empirical study on firebase," in 2021 IEEE/ACM 8th International Conference on Mobile Software Engineering and Systems (MobileSoft), 2021, pp. 56–60.

[13] NVD. Cve-2023-21492 detail. [Online]. Available: https://nvd.nist.gov/vuln/detail/cve-2023-21492

[14] ——. Cve-2023-30721 detail. [Online]. Available: https://nvd.nist.gov/vuln/detail/cve-2023-30721

[15] Google. Log info disclosure. [Online]. Available: https://developer.android.com/privacy-and-security/risks/log-info-disclosure

[16] H. Liang, D. Wu, J. Xu, and H. Ma, "Survey on privacy protection of android devices," in 2015 IEEE 2nd International Conference on Cyber Security and Cloud Computing, 2015, pp. 241–246.

[17] Z. Tan and W. Song, "Ptpdroid: Detecting violated user privacy disclosures to third-parties of android apps," in Proceedings of the 45th International Conference on Software Engineering, ser. ICSE '23. IEEE Press, 2023, p. 473–485. [Online]. Available: https://doi-org.ezproxy.rit.edu/10.1109/ICSE48619.2023.00050

[18] Y. Zhou and X. Jiang, "Dissecting android malware: Characterization and evolution," in 2012 IEEE Symposium on Security and Privacy, 2012, pp. 95–109.

[19] N. Cheng, X. Oscar Wang, W. Cheng, P. Mohapatra, and A. Seneviratne, "Characterizing privacy leakage of public wifi networks for users on travel," in 2013 Proceedings IEEE INFOCOM, 2013, pp. 2769–2777.

[20] A. Mylonas, M. Theoharidou, and D. Gritzalis, "Assessing privacy risks in android: A user-centric approach," in Risk Assessment and Risk-Driven Testing: First International Workshop, RISK 2013, Held in Conjunction with ICTSS 2013, Istanbul, Turkey, November 12, 2013. Revised Selected Papers 1. Springer, 2014, pp. 21–37.

[21] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," ACM Trans. Intell. Syst. Technol., vol. 16, no. 5, Aug. 2025. [Online]. Available: https://doi-org.ezproxy.rit.edu/10.1145/3744746

[22] P. Hacker, A. Engel, and M. Mauer, "Regulating chatgpt and other large generative ai models," in Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency, ser. FAccT '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1112–1123. [Online]. Available: https://doi-org.ezproxy.rit.edu/10.1145/3593013.3594067

[23] Z. Chen, S. S. Deo, P. C. R. Puttaparthi, Y. Tang, X. Zhang, and W. Shang, "From logging to leakage: A study of privacy leakage in android app logs," in Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2484–2485. [Online]. Available: https://doi.org/10.1145/3691620.3695340

[24] Android Developers, "Logcat command-line tool," https://developer.android.com/studio/debug/logcat, 2026, accessed: 2026-02-22.

[25] Google. Understanding-logging:security and privacy. [Online]. Available: https://source.android.com/docs/core/tests/debug/understanding-logging#security-privacy

[26] U. D. of Labor. Guidance on the protection of personally identifiable information (pii). [Online]. Available: https://www.dol.gov/general/ppii

[27] (2024) Cve. [Online]. Available: https://cve.mitre.org/

[28] NIST. Vulnerability metrics. [Online]. Available: https://nvd.nist.gov/vuln-metrics/cvss

[29] ——. Cve-2016-10135. [Online]. Available: https://nvd.nist.gov/vuln/detail/cve-2016-10135

[30] ——. Cve-2017-18140. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2017-18140

[31] Google. Android debug bridge (adb). [Online]. Available: https://developer.android.com/tools/adb

[32] X. Zhang, J. Heaps, R. Slavin, J. Niu, T. D. Breaux, and X. Wang, "DAISY: dynamic-analysis-induced source discovery for sensitive data," ACM Trans. Softw. Eng. Methodol., vol. 32, no. 4, pp. 89:1–89:34, 2023. [Online]. Available: https://doi.org/10.1145/3569936

[33] Android. Declare your app's data use. [Online]. Available: https://developer.android.com/privacy-and-security/declare-data-use

[34] forensics. forensics. [Online]. Available: https://forensics.spreitzenbarth.de/android-malware/

[35] R. Slavin, X. Wang, M. B. Hosseini, J. Hester, R. Krishnan, J. Bhatia, T. D. Breaux, and J. Niu, "Toward a framework for detecting privacy policy violations in android application code," in Proceedings of the 38th International Conference on Software Engineering, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 25–36. [Online]. Available: https://doi-org.ezproxy.rit.edu/10.1145/2884781.2884855

[36] Z. Yang, M. Yang, Y. Zhang, G. Gu, P. Ning, and X. S. Wang, "Appintent: analyzing sensitive data transmission in android for privacy leakage detection," in Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security, ser. CCS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 1043–1054. [Online]. Available: https://doi.org/10.1145/2508859.2516676

[37] Z. Yang and M. Yang, "Leakminer: Detect information leakage on android with static taint analysis," in 2012 Third World Congress on Software Engineering, 2012, pp. 101–104.

[38] Y. He, X. Yang, B. Hu, and W. Wang, "Dynamic privacy leakage analysis of android third-party libraries," J. Inf. Secur. Appl., vol. 46, pp. 259–270, 2019. [Online]. Available: https://doi.org/10.1016/j.jisa.2019.03.014

[39] Logpai. Loghub. [Online]. Available: https://github.com/logpai/loghub

[40] Y. Huo, Y. Su, C. Lee, and M. R. Lyu, "Semparser: A semantic parser for log analytics," in Proceedings of the 45th International Conference on Software Engineering, ser. ICSE '23. IEEE Press, 2023, p. 881–893. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00082

[41] Z. Li, C. Luo, T. Chen, W. Shang, S. He, Q. Lin, and D. Zhang, "Did we miss something important? studying and exploring variable-aware log abstraction," in 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023. IEEE, 2023, pp. 830–842. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00078

[42] A. Lyons, J. Gamba, A. Shawaga, J. Reardon, J. Tapiador, S. Egelman, and N. Vallina-Rodriguez, "Log: It's big, it's heavy, it's filled with personal data! measuring the logging of sensitive information in the android ecosystem," in 32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 2115–2132. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/lyons

[43] Z. Chen, V. Nava-Camal, T. Roy, Z. Li, Y. Tang, X. Zhang, and H. Yang, "Exploring large language models' potential for privacy leakage detection in android app logs: An empirical study," IEEE Software, vol. 43, no. 1, pp. 57–63, 2026.

[44] Y. Zhao, K. Yao, Y. Tang, and W. Shang, "An empirical study of privacy leakage vulnerability in third-party android logs libraries," in Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2026.